

DEMO

Backlog

A local-first task queue your AI coding agents can read and write directly

Mazin Ahmed · github.com/mazen160/backlog

THE WALL

You start a task in the evening. The agent reads the codebase, writes a plan, ships half the work.

You are tired. You close the tab.

The next morning you open the same agent, type **"where were we?"**

And it has no idea.

The plan, the comments, the decisions, the "we decided not to use Redis because of X" – gone. It lived in the conversation, and the conversation is gone.

THE REAL PROBLEM

The default model for AI coding agents is **one long chat thread**.

Everything the agent knows lives in the active context window.

WHAT HAPPENS	WHY IT HURTS
Hit context limit	300–500k tokens per session, same files re-read constantly
Switch IDEs	Claude Code $\neq$ Cursor – neither knows what the other did
Resume weeks later	No history of what was tried, rejected, or decided
Track in markdown	Doesn't scale, creates its own coordination burden

The fix is not better prompts. The fix is moving work state out of

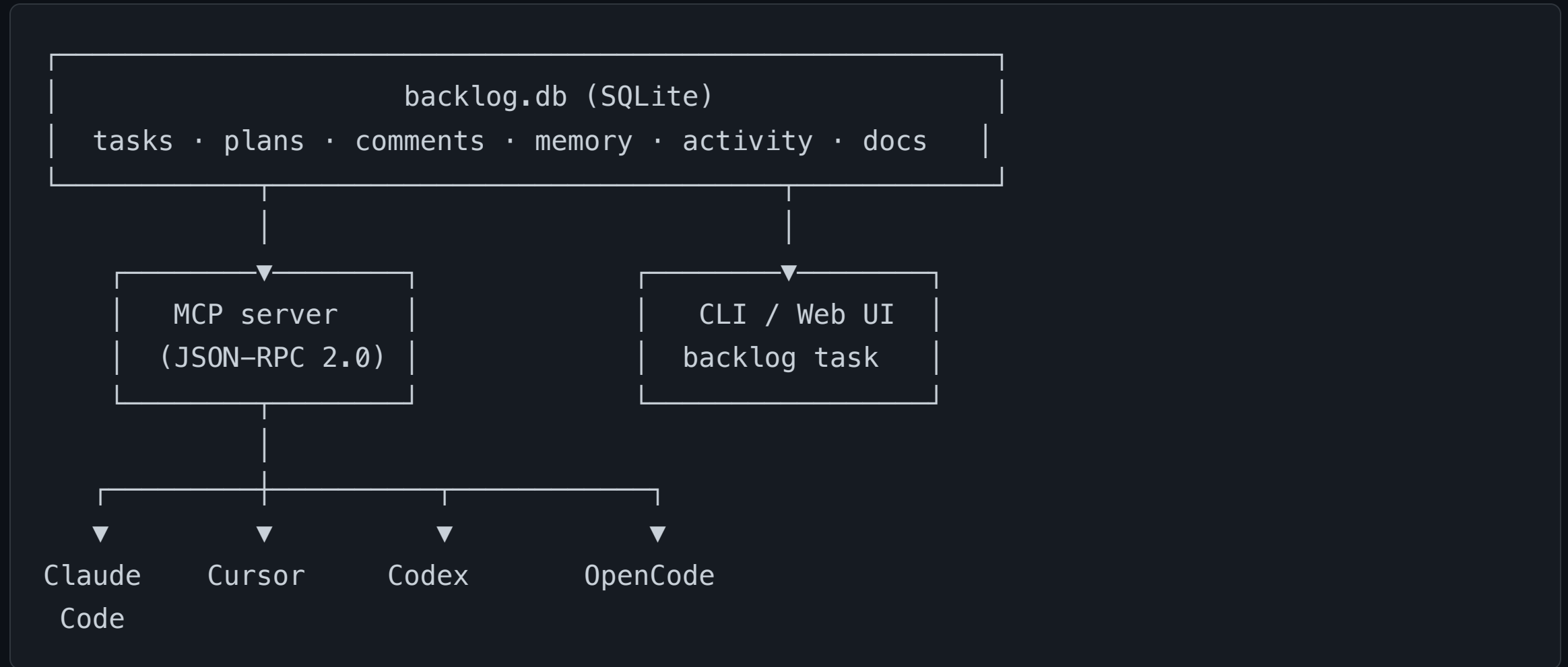
WHAT BACKLOG IS

A **single Go binary** that ships a SQLite-backed task queue with an MCP server built in.

```
backlog init          # creates backlog.db + config.toml next to your code
backlog install-skills # writes 4 agent skills into Claude Code, Cursor, Codex, OpenCode
```

From that point on, every agent you use points at the same database via MCP.

ARCHITECTURE



One database. Every session reads from it and writes to it.

WHAT SURVIVES BETWEEN SESSIONS

Five things that normally die when you close the chat tab:

	WHAT IT IS
Tasks	Title, description, type, status, priority, actor
Plans	Versioned markdown on each task – every edit is a new immutable version
Comments	Append-only, actor-attributed thread per task
Memory	Project-wide tagged notes: decisions, assumptions, lessons learned
Activity log	Every state transition, who made it, when

The IDE does not matter. The agent does not matter. The model does not matter. What matters is the database they all read and write.

MEMORY: THE PART WITH THE MOST IMPACT

Memory is how "we use JWT TTL of 15 minutes because of the March auth incident" gets preserved as a fact the next agent will consult before it touches anything auth-related.

```
# Agent finishes a task and writes a durable decision
backlog memory add \
  --project api \
  --body "JWT TTL = 15min. 2026-03 incident: stale tokens replayed against v2 endpoint." \
  --tag "decision,auth,security" \
  --as ai:claude-code
```

```
# Three weeks later, a different agent in a different IDE
backlog memory list --project api --tag auth
```

Memory is **mutable**, **tagged**, **queryable**, and **persists indefinitely**. It is the agent's working scratchpad across sessions

THE AGENTIC LOOP

The first three commands a fresh session runs:

```
backlog task list --status doing --json  
backlog memory list --project api --tag decision --json  
backlog activity --limit 20 --json
```

Then it picks a task and works.

There is no "where were we?" conversation. The agent reads its own queue, its own decisions, its own recent history, and starts working.

Pick → Plan → Ship → Review → Pick the next task.

ACTOR ATTRIBUTION

Every row in the database carries `actor_kind` and `actor_name`.

```
human:alice      created  TASK-42  "Fix login timeout"
ai:claude-code   updated  TASK-42  status → doing
ai:claude-code   added    PLAN-7   v2 "Revised: increase timeout to 30s, add test"
human:alice      commented                "Looks good. Ship it."
ai:claude-code   updated  TASK-42  status → done
```

```
backlog task list --actor-kind ai          # what agents did
backlog task list --actor-name claude-code # what Claude specifically did
backlog activity --actor-kind ai --limit 50
```

When something weird shows up, **you can answer "which agent did this?" instantly.**

ENGINEERING DECISIONS

Local-first, not cloud

The "server" is a SQLite file on your disk. No account. No API key. No telemetry. Reads take ~20ms. A SaaS round-trip takes 4 seconds – agents query the queue dozens of times per session.

Single binary, ~17 MB

No runtime. No dependencies. Runs on macOS, Linux, Windows – arm64 and amd64. `go install` and you are done.

MCP-native

JSON-RPC 2.0 over stdio. Plugs into Claude Code, Cursor, Codex, OpenCode without custom integrations. One protocol, every client.

4 embedded skills

`backlog`, `backlog-enhance-tasks`, `backlog-loop`, `backlog-goal` install in

WHAT I LEARNED

Task quality is the real bottleneck

A vague task burns 200k tokens before you get a useful PR.

A well-defined task ("Fix JWT validation in `internal/auth/jwt.go:84`. Add a regression test asserting 401 on an unsigned token.") finishes in under 30k tokens, on the first attempt.

Fresh subagents beat long threads

	TOKENS PER SESSION
Long-running thread	~500k
Fresh subagents (same work)	30–60k

A fresh agent pulls one task, plans it, ships it, and exits in a clean context window. **That is the order-of-magnitude cost reduction – and it is real**

TRY IT

```
go install github.com/mazen160/backlog/cmd/backlog@latest
```

```
backlog init
```

```
backlog install-skills
```

That installs the binary, creates a workspace, and writes four agentic-loop skills into Claude Code, Cursor, Codex, and OpenCode – whichever ones you have configured.

Or with curl:

```
OS=$(uname -s | tr '[:upper:]' '[:lower:]')  
ARCH=$(uname -m | sed 's/x86_64/amd64;/s/aarch64/arm64/')  
curl -L https://github.com/mazen160/backlog/releases/latest/download/backlog_${OS}_${ARCH}.tar.gz | tar xz  
sudo mv backlog /usr/local/bin/
```

LINKS

Repo	<code>github.com/mazen160/backlog</code>
Docs	<code>mazen160.github.io/backlog</code>
MCP setup	<code>mazen160.github.io/backlog/mcp.html</code>
License	MIT

If it saves you 20 minutes a day, that is roughly two weeks a year.
That is the bar I built it to clear.

Questions?